

Лабораторна робота N2

Класи та об'єкти.

Задача. Навчитись оголошувати класи. Навчитись створювати об'єкти з допомогою конструкторів та використовувати елементи класів. Примітивні та посилальні типи.

Завдання.

1. Обрати тему курсової роботи.

В ході курсової роботи ви повинні запрограмувати візуальну та інтерактивну модель якоїсь комп'ютерної гри або кінофільма або іншого твору, яка містить не менше трьох **макрооб'єктів**, які виконують роль контейнерів різного призначення. В моделі існують **мікрооб'єкти**, які створюються, знищуються, пересуваються (в т.ч. і самі по собі - без команд користувача) і взаємодіють між собою і з макрооб'єктами (див. Рисунок 1). **Універсальний об'єкт** – це об'єкт, який містить всі об'єкти. **Макрооб'єкти** повинні мати змогу містити декілька **мікрооб'єктів** одночасно, і дозволяти їм вільно заходити і виходити. Також і користувач повинен мати змогу за бажанням вводити або виводити обрані **мікрооб'єкти** з **макрооб'єктів**. **Мікрооб'єкти** повинні бути трьох рівнів, щоб демонструвати використання наслідування.

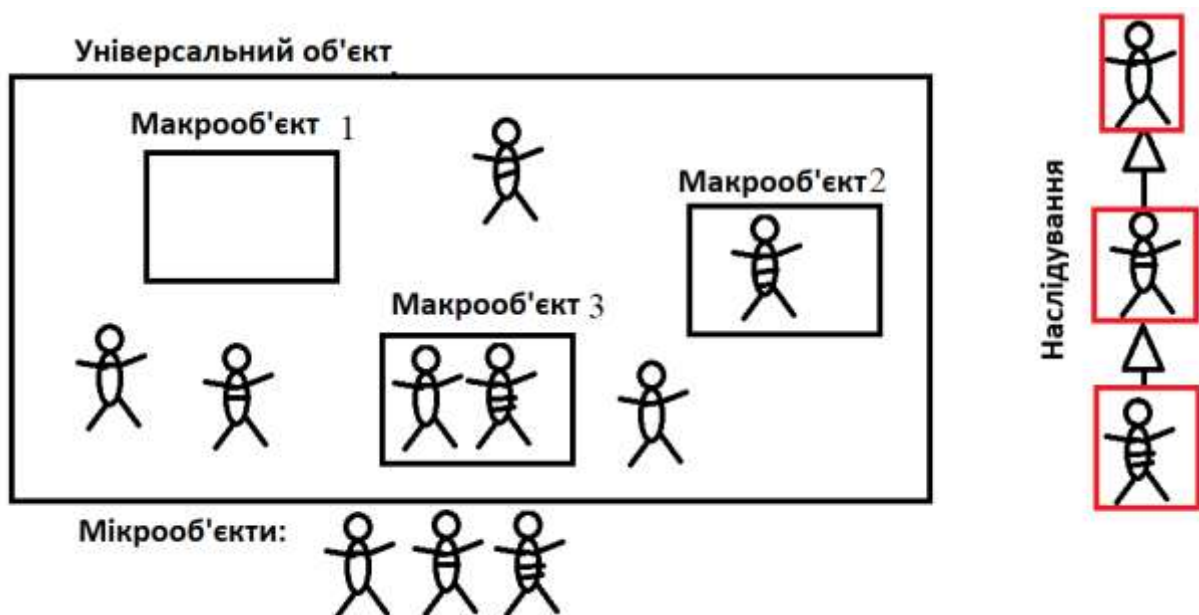


Рисунок 1 – Зовнішній вигляд моделі

Це загальний опис, а кожен студент обирає собі свою тему.

Наприклад, тема: Будівля ІТ-компанії. Макрооб'єкти: cubicle програміста, cubicle QA, серверна. Мікрооб'єкти: junior developer, middle developer, senior developer.

Ще приклад, тема: ВНТУ. Макрооб'єкти: аудиторія, бібліотека, буфет.

Мікрооб'єкти: студент-бакалавр, студент-магістр, аспірант.

Обравши тему, слід надіслати на емейл pz.vntu@gmail.com повідомлення, яке мстить наступні дані:

1. П.І.Б.
2. група
3. Телеграм акаунт (якщо є), наприклад <https://t.me/fuzzydik>
4. Username (Нікнейм акаунта Discord) або Display Name (Відображуване ім'я в текстовому каналі Discord), щоб викладач міг звернутись до студента в каналі Discord сервера через @Username або @Display Name.
5. Тема к.р., наприклад "Світ World of Warcraft. Макрооб'єкти: XXX1, XXX2, XXX3. Мікрооб'єкти: YYY1, YYY2, YYY3."

2. Оголосити клас мікрооб'єкту згідно варіанту курсової. Клас ОБОВ'ЯЗКОВО повинен містити не менше 4 елементів змінних. Як мінімум одна змінна повинна бути типу int, одна – типу double і як мінімум одна – рядок. Змінні-елементи класу мають бути приватними, для них повинні бути створені відповідні аксесори та мутатори, іншими словами – геттери та сеттери:

<https://www.baeldung.com/java-why-getters-setters>

3. По-перше, створити один конструктор, який ініціалізує об'єкти класу. По-друге, створити конструктор без аргументів, який викликає перший конструктор шляхом делегування (див. Using this with a Constructor отут

<https://docs.oracle.com/javase/tutorial/java/javaOO/thiskey.html>).

Конструктор повинен виводити на екран інформацію про те, що він був викликаний. Конструктори мають виводити інформацію про об'єкти, які вони створюють.

4. В оголошеному класі повинен бути присутній один статичний блок ініціалізації і один нестатичний.

<https://docs.oracle.com/javase/tutorial/java/javaOO/initial.html>

Обидва блоки повинні видавати на екран повідомлення про своє виконання.

5. В оголошеному класі повинні бути реалізовані функції `boolean equals(Object o)` та `String toString()`. В програмі повинно бути продемонстровано їх використання.

6. Окрім наведених вище функцій та конструкторів клас має містити 4 метода, які реалізують життєдіяльність об'єкта.

7. Повинна бути присутня функція `print`, яка виводить на екран стан об'єкта.

8. Об'єктів у програмі повинно бути створено не менше трьох – один, що є статичним елементом класу `Main` (Об'єкт 1), другий – звичайним елементом класу `Main` (Об'єкт 2), третій - локальним об'єктом функції `main` (Об'єкт 3).

9. Програма повинна запитувати у користувача, чи бажає він задавати параметри об'єктів, що створюються. Якщо бажання є, то користувач повинен мати змогу ввести значення створюваних об'єктів з консолі, інакше створювані об'єкти отримають значення за замовчуванням.

10. В програмі повинно бути реалізоване меню, в якому містяться наступні команди:

- вивести на екран Об'єкт 1;
- змінити параметри Об'єкта 1;
- вивести на екран Об'єкт 2;
- змінити параметри Об'єкта 2;
- вивести на екран Об'єкт 3;
- змінити параметри Об'єкта 3;

- обрати пару з трьох створених об'єктів і здійснити їх взаємодію;
- завершення програми.

11. На початку методу main повинно виводитись на екран повідомлення "Початок роботи", а перед завершуючим return з функції main - повідомлення "Кінець роботи". Це для того, щоб спостерігаючи за повідомленнями від main та конструкторів студенти могли скласти враження про порядок виклику конструкторів створюваних об'єктів.

Теоретичні відомості

//наступний проєкт ілюструє використання меню в консолі

```
import java.util.Scanner;
import java.util.Random;

public class Main{
    public static void main(String[] args){

        Random rnd = new Random();

        int guess = rnd.nextInt(100);

        int selection;
        Scanner input = new Scanner(System.in);

        while(true) {
            System.out.println("Ваш вибір:");
            System.out.println("-----\n");
            System.out.println("1 - Загадати число");
            System.out.println("2 - Спроба відгадати");
            System.out.println("3 - Вихід з програми");

            selection = input.nextInt();

            switch (selection) {
                case 1:
                    guess = rnd.nextInt(100);
                    break;
                case 2:
                    System.out.println("2 - Ваше число");
                    int effort = input.nextInt();
                    if (effort < guess) System.out.println("Ваше число менше за
загадане!");
                    else if (effort > guess) System.out.println("Ваше число більше за
загадане!");
                    else System.out.println("Ви відгадали!!!");
                    break;
                case 3:
                    System.exit(0);
            }
        }
    }
}
```

```

    }
}
}
}
}

```

```

/*

```

Наступний проект ми розглянемо на лекції. Він містить змінні-елементи класу, методи, конструктор по замовчуванню, та заміщення equals для порівняння ссилочного типу за змістом, об'єкт - статичний елемент класу Main, об'єкт - нестатичний елемент класу Main та об'єкти - локальні для функції Main.

Проект також ілюструє п'ять відмінностей примітивних типів від ссилочних типів.

```

*/

```

```

// Main.java
package com.company;

import java.util.Scanner;

public class Main {

    public static Scanner in;
    static { //Статичний блок ініціалізації
        in = new Scanner(System.in);
    }

    public static Student object1; //Об'єкт 1 з л.р.2
    static { //Статичний блок ініціалізації
        System.out.println("Static initialization block!");
        object1 = Student.askStudentParameters("Перший об'єкт - статичний елемент класу");
    }

    private Student object2; //Об'єкт 2 з л.р.2
    { //Нестатичний блок ініціалізації
        System.out.println("Non-static initialization block!");
        object2 = Student.askStudentParameters("Другий об'єкт - не-статичний елемент класу");
    }

    public static void change(int q)
    {
        ++q;
    }

    public static void change(Student st)
    {
        st.exam(4.0);

        st=new Student("Biden",5,12345);
        st.exam(5.0);
        st.exam(5.0);
    }
}

```

```
st.exam(5.0);
st.exam(5.0);
st.exam(5.0);
}
```

```
public static void main(String[] args) throws CloneNotSupportedException {
```

```
    //Student.university="Politeh";
```

```
    Main service= new Main();
```

```
    System.out.println("Доступ до другого об'єкту:"+service.object2);
```

```
    Student st1 = new Student("Obama", 1,23456); //Об'єкт 3 з л.р.2
    Student st2 = new Student("Trump", 3, 456789);
```

```
    st1.kurs(4);
    System.out.println(st1.kurs());
```

```
    st1.exam(4.5);
    st1.exam(5.0);
    st1.exam(5.0);
```

```
    st2.exam(3.0);
    st2.exam(3.5);
```

```
    Student.setStipendiaPorig(4.7); //Виклик статичного метода класа
```

```
    st1.print();
    st2.print();
```

```
    st1.promote();
    st2.promote();
    System.out.println("Після промоушена:");
```

```
    st1.print();
    st2.print();
```

```
    {
        int a = 100;
        System.out.println(a);
    }
```

```
    System.out.println(st1);
    System.out.println(st2);
```

```
    //1. 3 точки зору операції створення
    int a =15;
```

```
    Student s1;//=new Student("Bush",3, 4234225);
    s1=new Student("Bush",3, 245345);
    System.out.println(s1);
```

```
    //2. 3 точки зору операції присвоєння
    int b;
    b=a;
    System.out.println(a+" "+b);
    a=200;
    System.out.println(a+" "+b);
```

```

Student s2=new Student("Washington",4, 46363656);
//s1=s2;

s1 = (Student) s2.clone();

System.out.println("До зміни");
System.out.println(s1);
System.out.println(s2);

s1.exam(4.0);
s1.exam(5.0);

System.out.println("Після зміни");
System.out.println(s1);
System.out.println(s2);

//3. З точки зору операції порівняння
int c=100;
int d=100;

if( c==d ) System.out.println("c дорівнює d!");
else System.out.println("c НЕ дорівнює d!");

Student s3=new Student("Roosevelt", 5,232434);
Student s4=new Student("Roosevelt", 5,3667645);

if( s3==s4 ) System.out.println("==:s3 дорівнює s4!");
else System.out.println("==:s3 НЕ дорівнює s4!");

if( s3.equals(s4) )System.out.println("equals:s3 дорівнює s4!");
else System.out.println("equals:s3 НЕ дорівнює s4!");

//4. З точки зору передачі у функцію
int e=10;
System.out.println("Перед change" + e);
change(e);
System.out.println("Після change" + e);

Student s5 = new Student("Grant",3,464756756);
System.out.println("Перед change" + s5);
change(s5);
System.out.println("Після change" + s5);

//5. З точки зору масивів
int []primarr= new int[10];
int []primarr2= new int[] { 1,2,3,4,5};

for( int x : primarr) System.out.print(x+" ");
System.out.println();
for( int x : primarr2) System.out.print(x+" ");
System.out.println();

Student []refarr = new Student[10];
Student []refarr2 = new Student[] {new Student("Carter",2,3633535),
    new Student("Kennedy",3,242433),
    new Student("Lincoln",4,24242)};

for( Student s:refarr) System.out.println(s);

System.out.println();

for( Student s:refarr2) System.out.println(s);

```

```

    }
}

//Student.java
package com.company;

import java.util.Arrays;
import java.util.Objects;

enum Status{
    STUDENT, BAKALAVR, MAGISTR
}

public class Student implements Cloneable {
    private String name;
    private int kurs;
    private double [] zachotka;
    private int id;

    public static Student askStudentParameters(String msg)
    {

        System.out.println("Введіть параметри:"+msg);

        System.out.print("Ім'я:");
        String name=Main.in.nextLine();

        System.out.print("Курс:");
        String s=Main.in.nextLine();
        int kurs= Integer.parseInt(s);

        System.out.print("Номер зачетки:");
        s=Main.in.nextLine();
        int id= Integer.parseInt(s);

        System.out.println();

        return new Student(name, kurs, id);
    }

    public static String university = "VNTU";

    public String getName() // аксесор (getter)
    {
        return name;
    }

    public void setName(String n) // мутагор (setter)
    {
        name= n;
    }

    public int kurs() // аксесор (getter) R
    {
        return kurs;
    }

    public void kurs(int n) // мутагор (setter) W
    {
        kurs= n;
    }
}

```



```

@Override
public String toString() {
    return "Student{" +
        "name=" + name + "\" +
        ", kurs=" + kurs +
        ", zachotka=" + Arrays.toString(zachotka) +
        ", status=" + status +
        "}";
}

@Override
protected Object clone() throws CloneNotSupportedException {
    return super.clone();
}

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Student student = (Student) o;
    return kurs == student.kurs &&
        name.equals(student.name) &&
        status == student.status;
}

@Override
public int hashCode() {
    return Objects.hash(name, kurs, status);
}

public Student(String n, int kurs, int id ){

    name= n;
    this.kurs = kurs;
    this.id = id;

    zachotka= new double[0];

}

public Student()
{
    this("Іваненко", 1, 0); // Делегування конструкторів
}

/* Автоматично згенерований default конструктор
public Student() {
    super();
}
*/
public static double stipendiaPorig;

public static void setStipendiaPorig(double porig)
{
    stipendiaPorig=porig;
}

public static double promotionPorig;

static {

```

```

    promotionPorig=4.0;
}

Status status;

{
    status=Status.STUDENT;
}

public boolean promote() {

    if( zachotka==null )
    {
        System.out.println("Зачотка пуста!");
        return false;
    }

    for( double m:zachotka)
        if( m<promotionPorig )return false;

    switch(status)
    {
        case STUDENT: status=Status.BAKALAVR; break;
        case BAKALAVR: status=Status.MAGISTR; break;
        case MAGISTR:
        {
            System.out.println("Це остання доступна ступінь!");
            break;
        }
        default:
            System.out.println("Помилка! Неіснуючий статус!");
            break;
    }

    return true;
}

public void exam( double mark ){
    //Нарощування масива double [] на один елемент

    if( zachotka==null )zachotka=new double[1] ;
    else zachotka = Arrays.copyOf
        (zachotka,
            zachotka.length+1);

    zachotka[zachotka.length-1]= mark;
}

public void print() {
    System.out.printf("Студент %s (%s) Курс %d Університет %s Середній бал %f\n",
        this.name, status, kurs, university, serednijBal() );

    if(zachotka!=null ) {
        if (zachotka.length != 0) {
            for (int i = 0; i < zachotka.length; i++)
                System.out.printf("%3.1f ", zachotka[i]);
            System.out.println();
        }
        else System.out.println("Зачотка пуста!");
    }
    else System.out.println("Зачотка пуста!");
}

```

```

    if( serednijBal() < stipendiaPorig )
        System.out.println("Треба вчитись краще - нема стипендії!\n");
    else
        System.out.println("Отримує стипендію");
}

public double serednijBal() {
    double rezult=0.0;

    if( zachotka==null ) return 0.0;

    if(zachotka.length==0) return 0.0;

    for( int i=0; i<zachotka.length; i++ )rezult+=zachotka[i];

    return rezult/((double)zachotka.length);
}
}

```

```

//Наступний проєкт ілюструє використання статичних блоків ініціалізації
// https://vertex-academy.com/tutorials/ru/bloki-inicializacii-v-java-chast-1/
public class Main{
    static {
        System.out.println("This is first static block");
    }

    public Main(){
        System.out.println("This is constructor");
    }

    public static String staticString = "Static Variable";

    static {
        System.out.println("This is second static block and "
            + staticString);
    }

    public static void main(String[] args){
        Main statEx = new Main();
        Main.staticMethod2();
    }

    static {
        staticMethod();
        System.out.println("This is third static block");
    }

    public static void staticMethod() {
        System.out.println("This is static method");
    }

    public static void staticMethod2() {
        System.out.println("This is static method2");
    }
}

```

Наступний проект демонструє звичайне та форматване виведення на консоль, введення з клавіатури цілих чисел та рядків, конвертація int -> string та string -> int

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        // write your code here

        System.out.println("Hello world!");
        System.out.println("Bye world...");

        int i=10;
        int j=20;
        double q=10.345;

        System.out.printf("i=%d j=%d q=%3.2f\n",i,j,q);

        Scanner in = new Scanner(System.in);

        System.out.print("Input a number: ");
        int num = in.nextInt();

        System.out.printf("Your number: %d \n", num);

        String s = in.nextLine();
        System.out.println(s);

        s = Integer.toString(i, 16); //Преобразование int -> string
        System.out.println(s);

        s = "200";
        i=Integer.parseInt(s); //Преобразование string -> int
        System.out.printf("Converted300 number: %d \n", i);

    }
}
```

Контрольні питання

1. Що таке об'єкт?
2. Що таке стан об'єкту? Якими засобами мови Java він реалізується?
3. Що таке поведінка об'єкта? Якими засобами мови Java вона реалізується?
4. Що таке клас? Як оголошується клас у Java?
5. Проілюструйте співвідношення "клас-об'єкт"?
6. Що таке інкапсуляція? Наведіть приклад коду програми.
7. Що таке конструктор? Наведіть приклад коду програми.
8. Що таке конструктор по замовчуванню (default constructor) в Java?
9. Що таке статичний блок ініціалізації?
10. Що таке НЕстатичний блок ініціалізації?
11. Доступ до елементів з типом доступу private
12. Доступ до елементів, для яких не вказано модифікатор доступу (тобто тип доступу default)

13. Доступ до елементів з типом доступу `protected`
14. Чим відрізняється тип доступу `protected` у мові Java та C++?
15. Доступ до елементів з типом доступу `public`
16. Як здійснюється делегування конструкторів?
17. Як здійснюється перевантаження (`overloading`) конструкторів?
18. Як здійснюється перевантаження (`overloading`) функцій?
19. Що таке аксесор та мутатор?
20. Для чого використовується посилання `this` і звідки воно береться?
21. Що таке статичні елементи класу і чим вони відрізняються від НЕстатичних? Наведіть приклад коду програми.
22. Що таке перерахування (`enum`) і для чого воно використовується?
23. Як працює `foreach` модифікація конструкції `for` для масивів?
24. Як працює конструкція `switch`?
25. Як зробити осмислений вивод на консоль об'єкта функцією `System.out.println` ?
26. Звідки по замовчуванню береться функція `toString()`, яка перетворює об'єкт на рядок?
27. Як написати власну функцію `toString()`, яка перетворює об'єкт на рядок?
28. Роль класу `Object`
29. Що таке значені (примітивні, `value types`) типи? Наведіть приклади
30. Що таке посилальні (ссылочні, `reference types`) типи? Наведіть приклади
31. Відмінність примітивних типів від посилальних з точки зору операції створення
32. Відмінність примітивних типів від посилальних з точки зору операції присвоєння
33. Відмінність примітивних типів від посилальних з точки зору операції порівняння
34. Відмінність примітивних типів від посилальних з точки зору передачі у функцію у якості аргумента
35. Чим відрізняється передача аргумента у функцію за значенням від передачі аргумента у функцію за посиланням. Як передати аргумент за значенням у Java? Як передати аргумент за посиланням у Java?
36. Відмінність примітивних типів від посилальних з точки зору створення масиву
37. Відмінність примітивних типів від посилальних з точки зору місця їх зберігання та видимості
38. Як створити масив даних примітивного типу?
39. Як створити масив даних посилального типу?
40. Як порівнює об'єкти оператор `==`?
41. Як здійснити порівняння об'єктів не за адресою а їх вмістом?
42. Для чого використовується функція `public boolean equals(Object obj)` ?
43. Як порівняти два рядки `String`?
44. Що означає термін *прототип функції*?
45. Як у C++ створюються масиви першого типу?
46. Як у C++ створюються масиви другого типу?

47.Масиви якого типу присутні у Java?